

Introduction to Computing (CS 101)

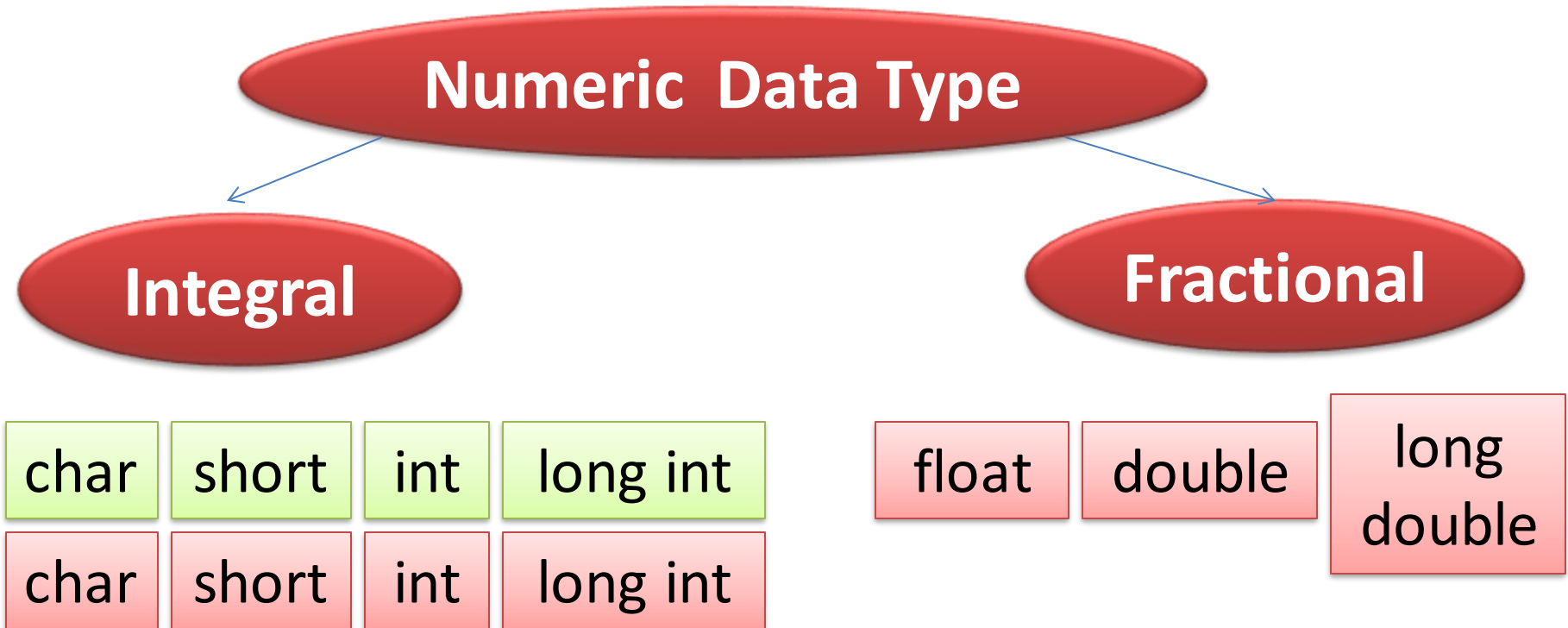
Statements, Operators and Expressions

T. Venkatesh & John Jose



Department of Computer Science & Engineering
Indian Institute of Technology Guwahati

Numeric Data Type



- `char, short, int, long int`
 - Signed and unsigned
- `float, double, long double`

C Statements

- Statements are terminated with a semicolon `;`

```
char acharacter;
```

```
int j = 18, k = -20;
```

```
printf("Initially, given j=18 and k=-20\n");
```

- Group of statements (compound statement) are enclosed by curly braces: `{` and `}`.
- Mark the start and the end of code block.

C Programming : Sum of A and B

```
#include <stdio.h>
```

```
int main() {
```

```
    int A,B, S;
```

```
    printf("Enter two  
           numbers ");
```

```
    scanf("%d %d", &A, &B);
```

```
    S=A+B;
```

```
    printf("Res=%d", S);
```

```
    return 0;
```

```
}
```

Start of the BLOCK

Statement 1

Statement 2

Statement 3

Statement 4

Statement 5

Statement 6

End of the BLOCK

Comments in C

- Single line of comment: `// comment here`
- More than single line of comment or expanded:
`/* comment(s) here */`
- Compiler exclude comment lines while translation
- Comments are used to add readability of the code

```
#include <stdio.h> // for printf()
/* main() function, where program
   execution starts */
int main() {
    /* declares variable and initializes it*/
    int i = 8;
    printf("value of i=%d\n", i);
    return 0;
}
```

Declaring Variables

- Before using a variable, we must give the compiler some information about the variable by declaring it.
- The declaration statement includes the data type of the variable.

- Examples of variable declarations:

□ **int** length ;

□ **float** area ;

- Visualization of the declaration

□ **int** length ;

FE07

Garbage value

length

- When we declare a variable
 - Space is set aside in memory to hold a value of the specified data type
 - That space is associated with the variable name
 - That space is associated with a unique **address**

Using Variables: Initialization

- Variables may be given initial values, or initialized, when declared.

```
int length=7;
```



length

7

```
float diameter=5.9;
```



diameter

5.9

```
char initial = 'A';
```



initial

'A'

Using Variables: Initialization

- Put initialized variables on a separate line
- Comment the initialization for better readability

– Example:

```
int height;    /* rectangle height */  
int width=6;  /* rectangle width  */  
int area;     /* rectangle area   */
```

```
int height, width = 6, area ;
```

[Not an error , but not a good practice]

Using Variables: Assignment

- Variables may have values assigned to them through the use of an assignment statement.
- This operator (=) does not denote equality.
- It assigns the value of the right-hand side of the statement (the expression) to the variable on the left-hand side.
- Only single variables may appear on the left-hand side of the assignment operator.
- Examples:

```
diameter = 5.9 ;
```

```
area = length * width ;
```

Arithmetic Operators in C

<u>Name</u>	<u>Operator</u>	<u>Example</u>
□ Addition	+	num1 + num2
□ Subtraction	-	initial - spent
□ Multiplication	*	fathoms * 6
□ Division	/	sum / count
□ Modulus	%	m % n

Division

- Integer division
 - If both operands of a division expression are integers, you will get an integer answer.
- The fractional portion is thrown away.
- Examples :

$$17 \quad / \quad 5 \quad = \quad 3$$

$$\square \quad 4 \quad / \quad 3 \quad = \quad 1$$

$$\square \quad 35 \quad / \quad 9 \quad = \quad 3$$

- Division by zero is mathematically undefined.
- If you allow division by zero in a program, it will cause a fatal error and program will terminate execution.

Division

- Float division
- Division where at least one operand is a floating point number will produce a floating point answer.
- Examples:

$$17.0 \quad / \quad 5 \quad = \quad 3.4$$

□ $4 \quad / \quad 3.2 \quad = \quad 1.25$

□ $35.2 \quad / \quad 9.1 \quad = \quad 3.86813$

- The integer operand is temporarily converted to a floating point, then the division is performed.

Modulus

- The expression $m \% n$ yields the integer remainder after m is divided by n .
- Modulus is an integer operation, both operands must be integers.
- Examples :

$$17 \% 5 = 2$$

$$\square \quad 6 \% 3 = 0$$

$$\square \quad 9 \% 2 = 1$$

$$\square \quad 5 \% 8 = 5$$

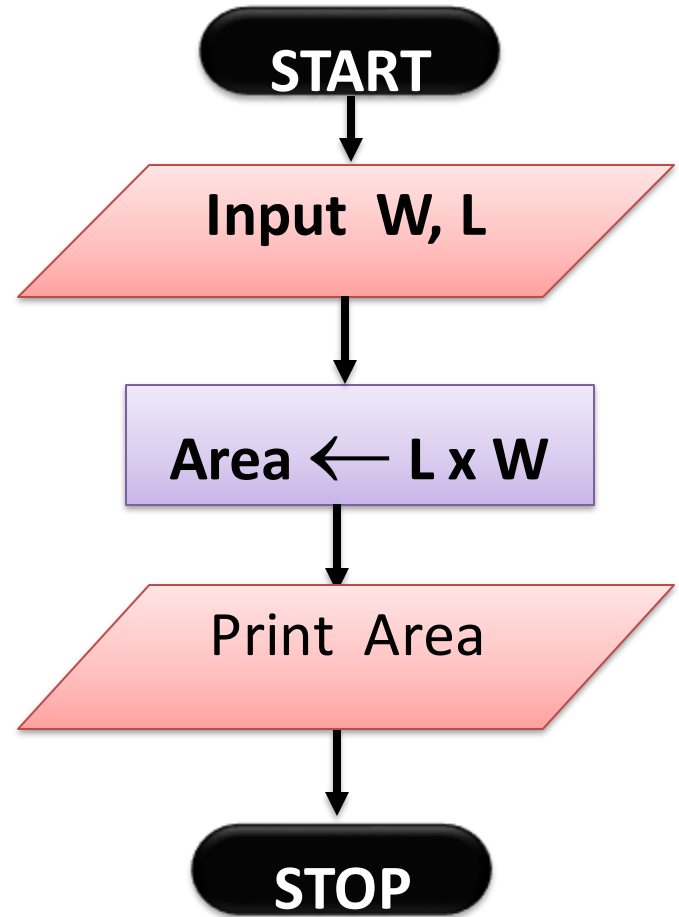
- Used to determine if an integer value is even or odd

$$\square \quad 5 \% 2 = 1 \quad \text{odd} \qquad 4 \% 2 = 0 \quad \text{even}$$

C Example 1: Area of Rectangle

```
#include <stdio.h>
```

```
int main() {  
    int L, W, Area;  
    printf("Enter L & W ");  
    scanf("%d %d", &L, &W);  
  
    Area=L*W;  
  
    printf("Area=%d", Area);  
    return 0;  
}
```

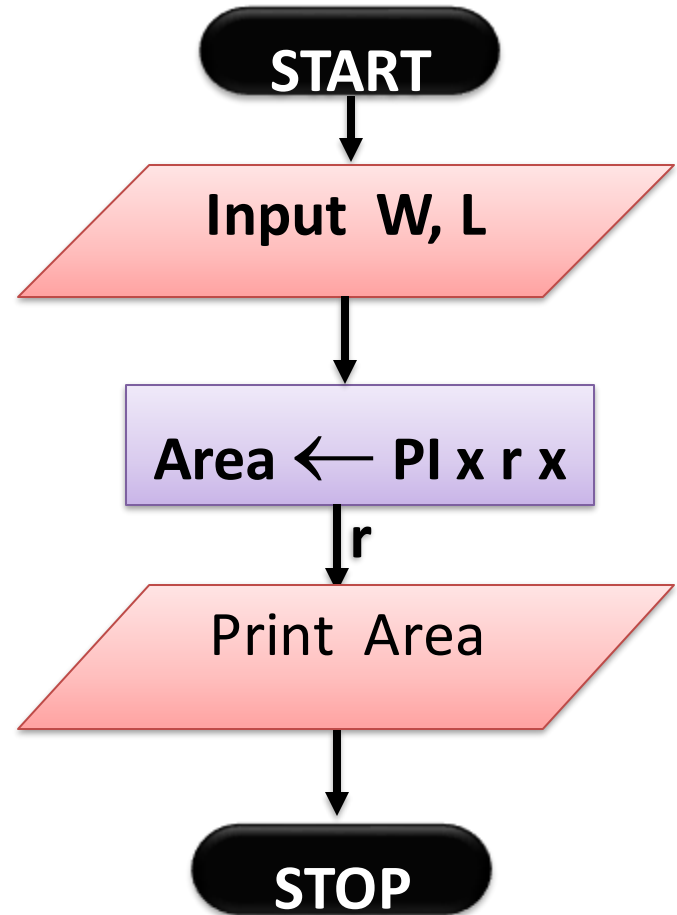


C Example 2: Area of Circle

```
#include <stdio.h>
#define PI 3.142
int main() {
    float r, Area;
    printf("Enter radius");
    scanf("%f", &r);

    Area=PI*r*r;

    printf("Area=%f", Area);
    return 0;
}
```

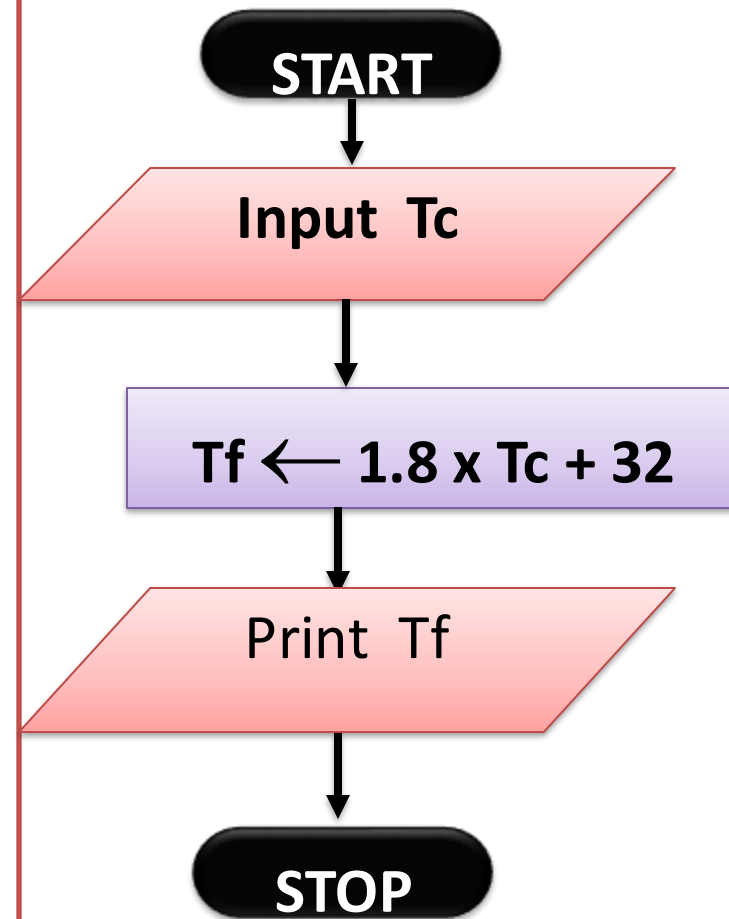


The literal **PI** value get replaced by 3.142

C Example 3: Temp Conversion

```
#include <stdio.h>
```

```
int main() {  
    float Tc, Tf;  
    printf("Enter Tc");  
    scanf("%f", &Tc);  
  
    Tf = (1.8 * Tc) + 32;  
  
    printf("Tf=%f", Tf);  
    return 0;  
}
```

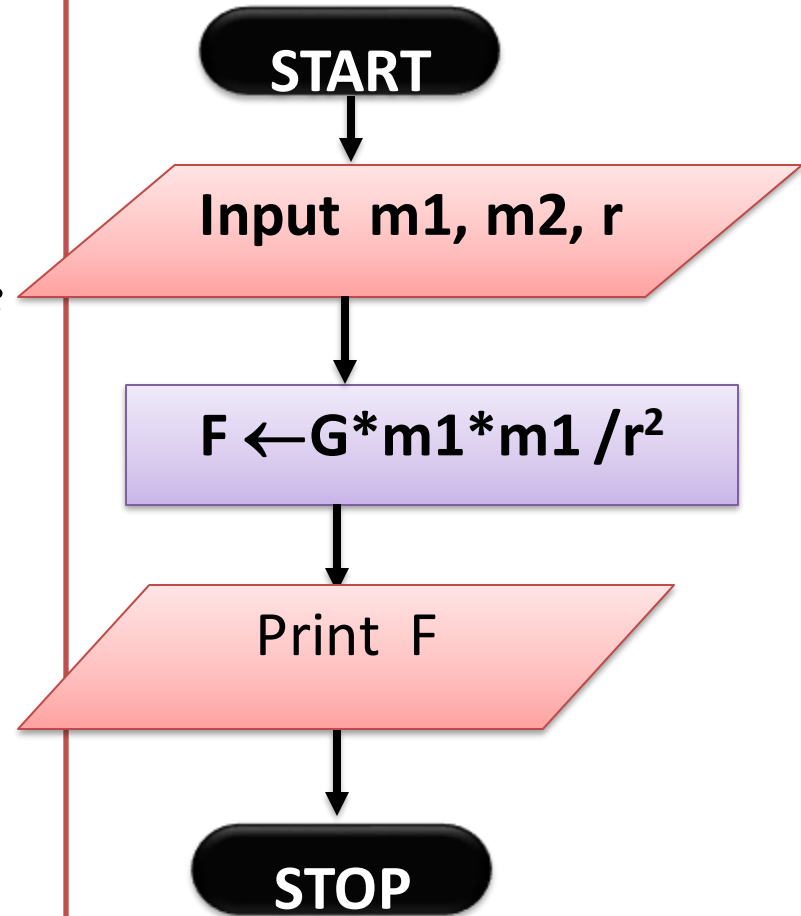


C Example 4: Force Between Two bodies

```
#include <stdio.h>
int main() {
    float F, m1, m2, r;
    float G=6.673e-11;
    printf("Enter m1 m2");
    scanf("%f %f", &m1, &m2);
    printf("Enter r");
    scanf("%f", &r);

    F = (G*m1*m2) / (r*r);

    printf("Tf=%f", F);
    return 0;
}
```



Expression Evaluation

- B-E-DM-AS Rule
- B : Bracket or Parenthesis ()
 - Only () used for expression
 - Curly braces {}, and square bracket [] used for some other purpose.
- E : Exponentiation or Order (O)
- DM: Division and Multiplication
- AS : Addition and Subtraction

Arithmetic Operators Precedence Rule

<u>Operator(s)</u>	<u>Precedence & Associativity</u>
$()$	Evaluated first. If nested (embedded), innermost first.
$* \quad / \quad \%$	Evaluated second. If there are several, evaluated left to right.
$+ \quad -$	Evaluated third. If there are several, evaluated left to right.
$=$	Evaluated last, right to left.

Expression Evaluation

- Evaluate $8+3*4/2$
 - DM have higher priority as compared to AS
 - All DM get evaluated left to right

$$8+\mathbf{3*4}/2 = 8+\mathbf{12/2} = 8+6 = 14$$

- Evaluate $15-(6+1)+30/(3*2)$

$$15-\mathbf{(6+1)}+30/\mathbf{(3*2)} = 15-\mathbf{(7)}+\mathbf{30/(6)}$$

$$\mathbf{15-7}+5=8+5=13$$

- Evaluate $(95/19)^2+3$

$$-\mathbf{(95/19)}^2+3 = \mathbf{(5)}^2+3 = 25+3 = 28$$

Using Parentheses

$$a + b * c$$

- Would multiply $b * c$ first, then add a to the result.
- Use parentheses to change the order in which an expression is evaluated.
- If you really want the sum of a and b to be multiplied by c , use parentheses to force the evaluation to be done in the order you want.

$$(a + b) * c$$

Evaluating Expressions

□ Given integer variables a, b, c, d, and e, where $a = 1$, $b = 2$, $c = 3$, $d = 4$, evaluate the following expressions:

□ $a + b - c + d$

□ $a * b / c + d$

□ $1 + a * b \% c$

□

Evaluating Expressions

□ Given integer variables a, b, c, d, and e, where a = 1, b = 2, c = 3, d = 4, evaluate the following expressions:

□ $a + b - c + d = 3 - 3 + 4 = 0 + 4 = 4$

□

□ $a * b / c + d = 2 / 3 + 4 = 0 + 4 = 4$

□

□ $1 + a * b \% c = 1 + 2 \% 3 = 1 + 2 = 3$

□

Increment and Decrement Operators

- The **increment operator** ++
- The **decrement operator** --
- Precedence: lower than (), but higher than * / and %
- Associativity: right to left
- Increment and decrement operators can only be applied to variables, not to constants or expressions
- If we want to add one to a variable, we can say:
□ `count = count + 1 ;`
 `count++ ;` OR `++count ;`
- Both do the same thing. They change the value of count by adding one to it.

Post-Increment Operator

- The position of the ++ determines when the value is incremented. If the ++ is after the variable, then the incrementing is done last (a **post-increment**).

```
□    int amount, count ;
```

```
□    count = 3 ;
```

```
□    amount = 2 * count++ ;
```

- amount gets the value of $2 * 3$, which is 6, and then 1 gets added to count.
- After executing the last line, amount is 6 and count is 4.

Pre-Increment Operator

- If the ++ is before the variable, then the incrementing is done first (a pre-increment).

□ `int amount, count ;`

□ `count = 3 ;`

□ `amount = 2 * ++count ;`

- 1 gets added to count first, then amount gets the value of $2 * 4$, which is 8.
- After executing the last line, amount is 8 and count is 4.

Example

□ **int** ans, val= 4 ;

□ Code

Val

Ans

4

garbage

□ val = value + 1 ;

□ val++ ;

□ ++val ;

□ ans = 2 * val++ ;

□ ans = ++val / 2 ;

□ val-- ;

□ --val ;

□ ans = --val * 2 ;

□ ans = val-- / 3 ;

Example

□ `int ans, val= 4 ;`

□ Code

Val

Ans

4

garbage

□ `val = val + 1 ;`

5

□ `val++ ;`

6

□ `++val ;`

7

□ `ans = 2 * val++ ;`

8

14

□ `ans = ++val / 2 ;`

9

4

□ `val-- ;`

8

□ `--val ;`

7

□ `ans = --val * 2 ;`

6

12

□ `ans = val-- / 3 ;`

5

2

Practice Question 1

□ Given `int a = 1, b = 2, c = 3 ;`

□ What is the value of this expression?

□ `++a * b - c--`

□ What are the new values of a, b, and c?

□ Expression: `++a * b - c--`

□ Value = `1`

□ What are the new values of a, b, and c?

□ `a = 2 b = 2 c = 2`

Practice Question 2

□ Given `int a =1, b =2, c =3, d =4 ;`

□ What is the value of this expression?

□ `++b / c + a * d++`

□ What are the new values of a, b, c, and d?

□ Expression: `++b / c + a * d++`

□ Value = `1+4 =5`

□ What are the new values of a, b, c, and d?

□ `a=1, b=3, c=3, d=5`

Assignment Operators

□ = += -= *= /= % =

□ Statement Equivalent Statement

□ `a = a+2 ;` `a += 2 ;`

□ `a = a-3 ;` `a -= 3 ;`

□ `a = a*2 ;` `a *= 2 ;`

□ `a = a/4 ;` `a /= 4 ;`

□ `a = a%2 ;` `a %= 2 ;`

□ `b = b+(c+2) ;` `b += c + 2 ;`

□ `d =d*(e-5) ;` `d *= e - 5 ;`

□ Example:

□ `int i = 1, j = 2, k = 3, m = 4 ;`

□ What is the value of `i += j + k * m`?

□ Ans: **i=15**

Thanks